

Clase 04 (04/14/2026)

1. Entorno, Espacio de trabajo y Paquetes de ROS

1.1. Entorno de ROS

Dos niveles

- *underlay*: Paquetes de ROS2 instalados (`/opt/ros/jazzy/*`). Contiene las dependencias del *overlay*.
- *overlay*: Entorno de desarrollo donde se programan los nuevos paquetes (`ros_ws/*`). Sobre-escribe paquetes del *underlay*.

1.2. Espacio de trabajo

Estructura de carpetas que contiene paquetes

- **src**: Donde se encuentra el código fuente (crear y editar paquetes)
- **build**: Archivos intermedios de compilación
- **install**: Los paquetes o *targets* instalados
- **log**: Información de debug

1.3. Paquete

Contenedor o marco del código. Contiene (o puede contener) código fuente, archivos launch, archivos de configuración, definición de mensajes, datos, documentación, etc. La estructura permite instalar el propio código y compartirlo de forma rápida. Archivos mínimos para paquetes **Python**

- **package.xml**: información acerca del paquete
- **setup.py**: instrucciones para instalar el paquete
- **setup.cfg**: usado por ROS para encontrar los ejecutables (en el caso de que existan)
- **/<nombre_paquete>**: Carpeta con el mismo nombre del paquete junto con el archivo `__init__.py` utilizado por las herramientas de ROS2

1.4. Uso de nombres

- de paquete: en estructura de archivos, **package.xml**, **setup.cfg**, **setup.py**, línea de comando
- de archivo: en estructura de archivos y **setup.py**
- de ejecutable: **setup.py** y línea de comando
- de nodo: código y línea de comando

2. rclpy: ROS Client Library para Python

Librería que permite el acceso a los elementos de ROS desde la sintaxis de Python utilizando tipos y patrones nativos. Componentes principales de la librería: Inicialización y *spinning*. Gestión de nodos, Topics, Servicios. Subscriber/Publisher. Parámetros y Logging (así como servicios, acciones y elementos de sincronización)

Documentación completa en: docs.ros2.org/latest/api/rclpy/index.html

2.1. Partes básicas de un programa

- Inicialización: `rclpy.init(..)` Debe ser llamado antes de cualquier otra función de ROS. Define el contexto.
- Creación de 1 o más nodos: `rclpy.create_node(..)` El nodo es el punto de acceso al sistema de ROS (topics, servicios, parámetros, etc).
- Procesamiento de tareas (*spinnign*): `rclpy.spin(..)` Procesar los callbacks y demás rutinas
- Apagado: `rclpy.shutdown()`

2.2. Manejo de nodos

- Crear un publisher: `node.create_publisher(..)` Tipo de mensaje, nombre del topic
- Crear un subscriber: `node.create_subscription(..)` Tipo de mensaje, nombre del topic, callback
- Timer: `node.create_timer(..)` Período, callback. Para realizar tareas en un período deseado.
- Loggear: `node.get_logger()` Mensajes en consola

3. Callbacks

Porción de código que es pasada como argumento, a otra parte de código, que se espera sea ejecutado (*call back*) en un momento conveniente.

Dependencias, ejecutable y compilación

■

4. Laboratorio

4.1. Creación del paquete

Creación del workspace

```
mkdir -p ros_ws/src
cd ros_ws
colcon build
```

Creación del paquete en Python

```
cd src
ros2 pkg create --build-type ament_python <nombre_paquete>
```

package.xml

Completar descripción, licencia, y maintainer.

```
<?xml version="1.0"?>
<?xml-model
  href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>

<package format="3">
  <name>nombre_paquete</name>
  <version>1.0.0</version>
  <description>descripción</description>
  <maintainer email="user@todo.todo">user</maintainer>
  <license>LICENCIA</license>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

setup.py

```
from setuptools import setup

package_name = 'nombre_paquete'

setup(
    name=package_name,
    version='1.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='user',
    maintainer_email='user@todo.todo',
    description='descripción',
```

```

license='LICENCIA',
tests_require=['pytest'],
entry_points={
    'console_scripts': [
        'nombre_ejecutable = nombre_paquete.nombre_archivo_fuente:main'
    ],
},
)

```

setup.cfg

```

[develop]
script_dir=$base/lib/nombre_paquete
[install]
install_scripts=$base/lib/nombre_paquete

```

4.2. Programación nodo publisher

```

import rclpy
# Importar el tipo de mensaje String
from std_msgs.msg import String

def main(args=None):
    # 1. Inicialización
    rclpy.init(args=args)

    # 2. Creación de nodo
    nodo = rclpy.create_node('publicador')    # Nombre del nodo

    # 2.1 Creación de publisher: Tipo=String, Nombre='chat', Queue=10
    pub = nodo.create_publisher(String, 'chat', 10)

    # 2.2 Creación de mensaje
    msg = String()    # Crear un mensaje de tipo String (std_msgs/msg/String)
    i = 0

    # 2.3 Programación de la función de callback que será llamada por el timer
    def timer_callback():
        nonlocal i
        # Rellenar el contenido del campo 'data' del mensaje
        msg.data = f'Mensaje de prueba {i}'
        # Publicar el mensaje
        pub.publish(msg)

        # Escribir en la terminal a modo de debug
        nodo.get_logger().info(f'Publicando: "{msg.data}"')
        i += 1

    # Cada medio segundo publicar un mensaje
    timer_period = 0.5 # seconds
    # 2.4 Creación del timer
    timer = nodo.create_timer(timer_period, timer_callback)

    # 3. Procesamiento de mensajes y callback
    rclpy.spin(nodo)

    # 4. Finalización

```

```

    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

4.3. Compilar y ejecutar

Volver a la carpeta `ros_ws`

Instalar dependencias

```
rosdep install -i --from-path src --rosdistro jazzy -y
```

Compilar

```
colcon build --packages-select <nombre_paquete> --symlink-install
```

Source y ejecutar

```

source install/setup.bash
ros2 run <nombre_paquete> <nombre_ejecutable>

```

4.4. Programación nodo subscriber

```

import rclpy
# Importar el tipo de mensaje String
from std_msgs.msg import String

def main(args=None):
    # 1. Inicialización
    rclpy.init(args=args)

    # 2. Creación de nodo
    nodo = rclpy.create_node('suscriptor')

    # 2.1 Programación de función de callback
    def sub_callback(msg):
        # Mostrar el mensaje en consola
        nodo.get_logger().info(f'Recibí: "{msg.data}"')

    # 2.2 Creación de suscriptor
    sub = nodo.create_subscription(String, 'chat', sub_callback, 10)

    # 3. Procesamiento de mensajes y callback
    rclpy.spin(nodo)

    # 4. Finalización
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Será necesario incluir el nuevo ejecutable en `setup.py`